

一.基础内容

0.基本单位

1Byte(字节)=8bit(位)

1.进制转换

十进制转任意进制：整数部分除r取余，逆序；小数部分乘r取整，顺序。
任意进制转十进制：整数部分按权展开，逆序；小数部分按权展开，顺序。

1. MSB (Most Significant Bit)：指的是一个二进制数字中最左边的位，也就是具有最高权重的位。
LSB (Least Significant Bit)：指的是一个二进制数字中最右边的位，也就是具有最低权重的位
2. 二进制转十进制：除2取余，余数自高到低排列，再逆序。例：P33
3. 十进制小数转二进制：乘2取整，取整得到的0或1自高到低排列。例：P34
4. 二进制转八进制：3位一组，从右到左，不足补0。反之亦然
5. 二进制转十六进制：4位一组，从右到左，不足补0。反之亦然

2.原码、反码、补码

0.
 - 1's complement, 2's complement（反码/补码）对应无符号数，指的是一种操作，1's complement指的是按位取反，2's complement指的是按位取反+1；
 - 1's complement form, 2's complement form（反码形式/补码形式）对应有符号数；反码/补码时符号位不变

下列内容针对无符号数/反补码

1. 反码：每一位按位取反
2. 补码：反码+1

下列内容针对有符号数/反补码形式

1. 正数的原码、反码、补码都相同。
2. 负数的反码：符号位不变，其余位取反。
3. 负数的补码：反码+1。
4. 0的反码、补码都是0。

3.单精度浮点二进制数（P42）

没有那么重要

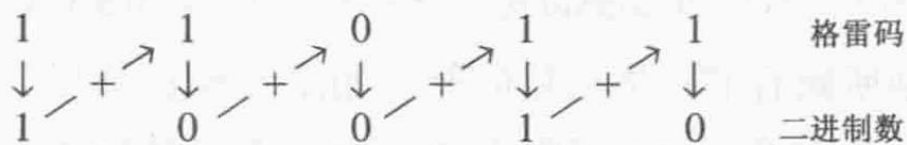
4A.BCD码（P55）

- 默认为8421BCD码
- 用四位二进制数表示一位十进制数
- 开头结尾的0不能去掉
- 二进制用不完，会出现无关项
- 任意进制数转BCD码都需要先转换成十进制数

4B.格雷码（无权码）（P58）

任意两个相邻的数只有一位不同
转二进制：错位做异或，最高位不变

例如，格雷码 11011 到二进制的转换过程如下：



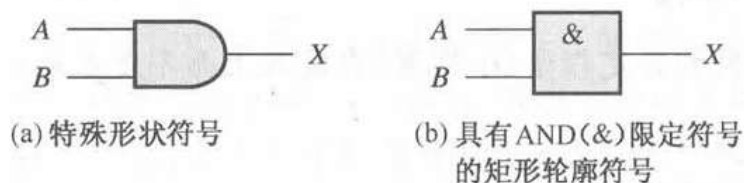
二进制数是 10010。

4C. 余3码

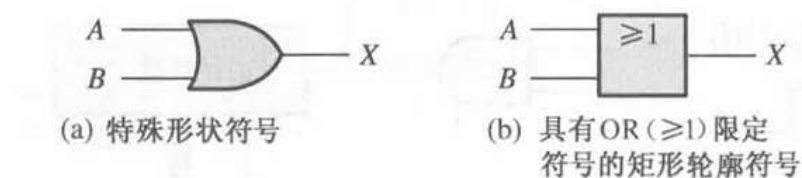
将每个十进制数位转换为对应的二进制数之前，先将该十进制数值加上3。例如，十进制数0在余3码中表示为0011（即二进制的3），而十进制数9在余3码中则表示为1100（即二进制的12）。

二. 逻辑门

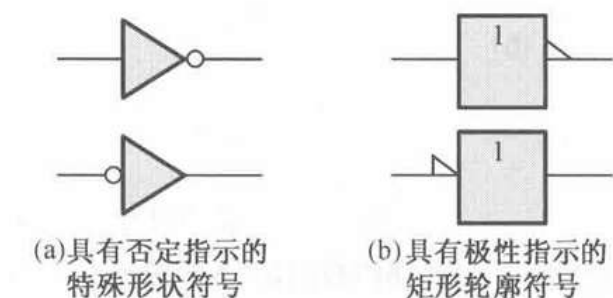
1. 与门 and ×



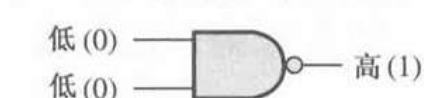
2. 或门 or +



3. 非门 not !



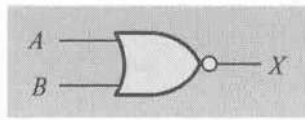
4. 与非门 NAND



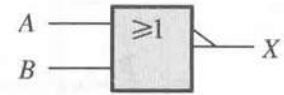
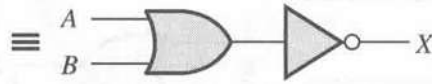
通用性：P170

与非和非或可以通过德摩根定律相互转换

5. 或非门 NOR



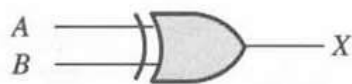
(a) 特殊形状符号，2 输入或非门和等价的或门+非门



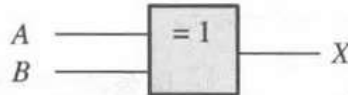
(b) 矩形轮廓符号，具有极性指示的 2 输入或非门

与或非与可以通过德摩根定律相互转换

6. 异或门 XOR $\oplus$ 不同为1，相同为0



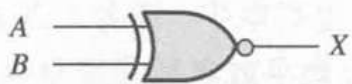
(a) 特殊形状符号



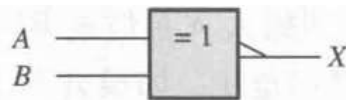
(b) 矩形轮廓符号

图 3.42 异或门的标准逻辑符号

7. 同或门 XNOR 相同为1，不同为0



(a) 特殊形状符号



(b) 矩形轮廓符号

图 3.45 同或门的标准逻辑符号

8. 通用逻辑门(Universal Gate)

- single与非门(NAND):

5.3.1 与非门作为通用的逻辑元件

◇ 与非门可以用来产生任意的逻辑函数。

与非门是一种通用门，因为它可以用来产生反相、与、或和或非的函数。反相器可以通过把与非门的所有输入连接在一起而形成，如图 5.18(a) 给出的 2 输入与非门，在效果上相当于一个输入。一个与门可以仅由与非门产生，如图 5.18(b) 所示。一个或门可以仅由与非门产生，如图 5.18(c) 所示。最后，一个或非门的产生如图 5.18(d) 所示。

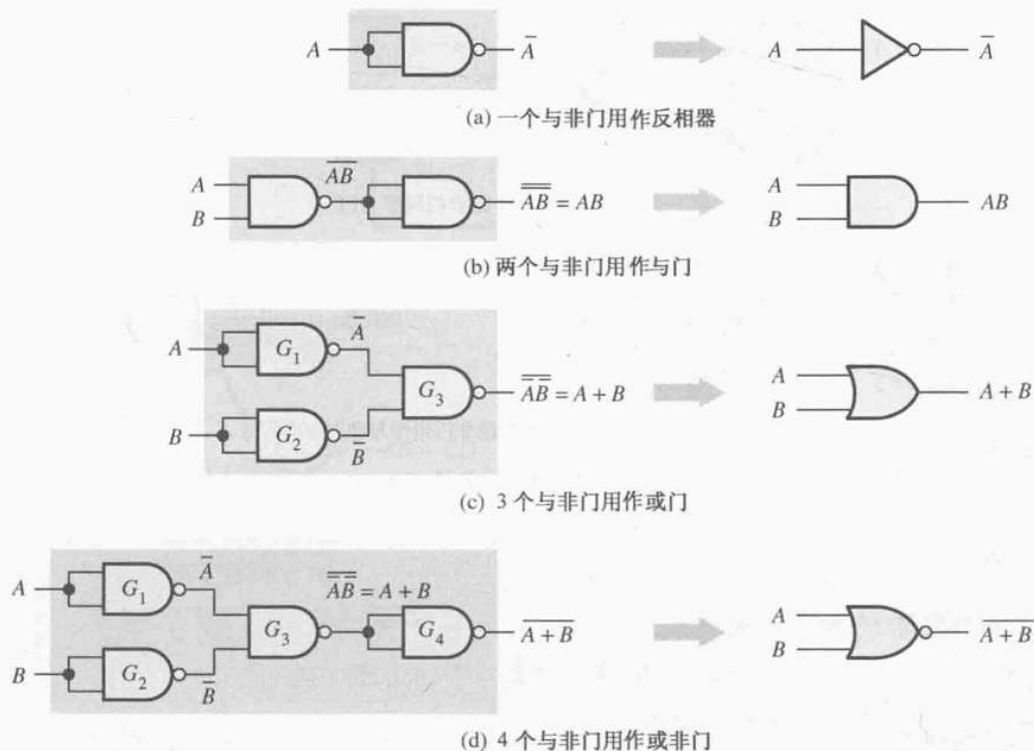


图 5.18 与非门的通用应用(打开文件 F05-18 可验证该操作)

MultiSim

- single或非门(NOR):

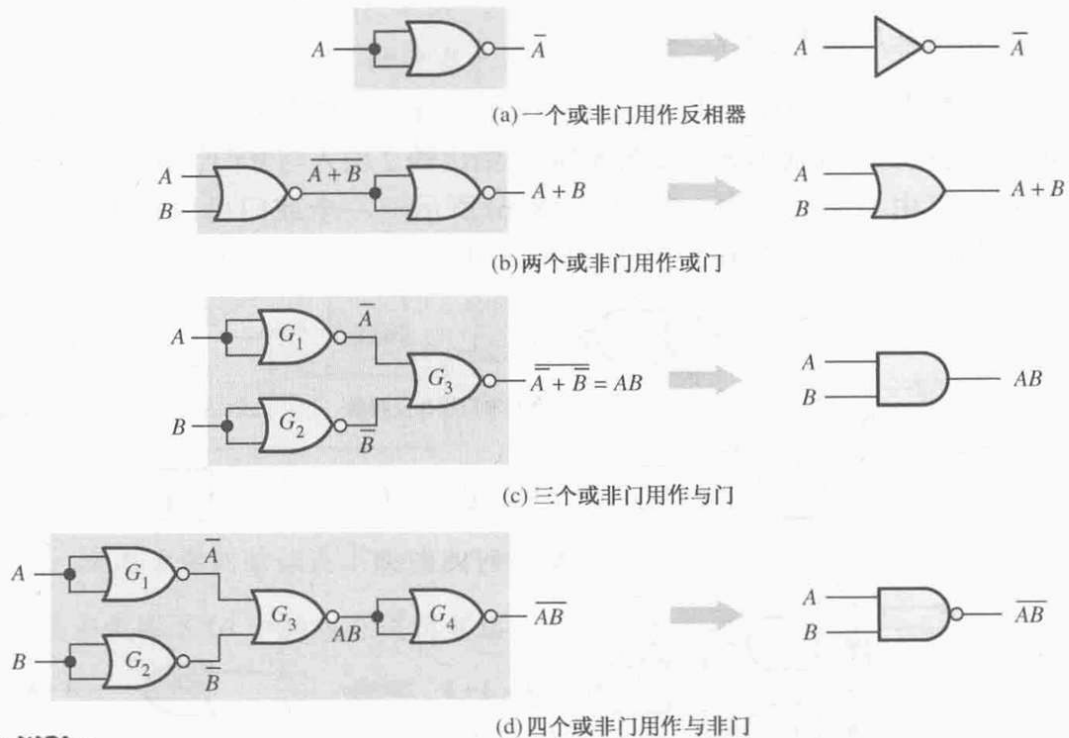
5.3.2 或非门作为通用的逻辑元件

◇ 或非门可以用来产生任意的逻辑函数。

如同与非门，或非门可以用来产生反相、与、或和与非的函数。一个非电路或反相器可以通过把或非门的所有输入连接在一起而形成，如图 5.19(a) 所给出的 2 输入或非门，在效果上相当于一个输入。同样，一个或门可以由或非门产生，如图 5.19(b) 所示。一个与门可以使用或非门构造出来，如图 5.19(c) 所示。这里或非门 G_1 和 G_2 用作反相器，最后的输出应用德·摩根定理推导如下：

$$X = \overline{\overline{A} + \overline{B}} = AB$$

图 5.19(d) 给出了或非门如何形成与非门电路。



- 与门(AND)+非门(NOT)
- 或门(OR)+非门(NOT)

三·布尔代数

1. 运算律

1. 交换律
2. 结合律
3. 分配律
4. 德摩根定律: $(XY)' = X' + Y'$, $(X + Y)' = X'Y'$
5. 吸收律: $A + AB = A$, $A + A'B = A + B$, $(A + B)(A + C) = A + BC$
6. 邻接律: $XY + XY' = X$, $(X + Y)(X + Y') = X$

2. 表达式形式

1. 积之和 SOP → 最小项/标准积之和: 每一个积项都包含所有变量
P.S. 最小项编码中不同变量顺序表示形式不一样。比如同为 $A'B$, $F(A, B) = m(0, 1) = m_1$ 和 $F(A, B) = m(1, 0) = m_2$ 是不同的。
2. 最小项: 对于一个给定的布尔函数，如果某个特定输入组合的结果是 1，则这个输入组合被称为该函数的一个最小项。例如，在三变量函数 $F(A, B, C)$ 中，如果当 $A=1, B=0, C=1$ 时 $F=1$ ，那么对应的最小项就是 $A+B'+C$ 。最大项同理。对应真值表的输出结果为 1

3. 和之积POS->最大项/标准和之积：每一个和项都包含所有变量，**对应真值表的输出结果为0**
4. 最小项和最大项的转换（P131）：m里除了SOP就是POS。举个例子：
对三变量ABC：最小项m(0,1,2,4,5,7)对应最大项M(3, 6, 8)。
5. 布尔表达式转真值表：
 - 1.POS/最大项/和之积：在布尔表达式中找0，比如 $ABC+AB'C$ ，对应的是真值表中 $A=0,B=0,C=0$ 的那一行。对应输出也为0。
 - 2.SOP/最小项/积之和：在布尔表达式中找1，比如 $ABC'+AB'C$ ，对应的是真值表中 $A=1,B=1,C=1$ 的那一行。对应输出也为1。

3.卡诺图（P135）

和上面的真值表一样，SOP填入0，POS填入1。

1. 合并规则：画圈
 - 1.相邻的1可以画入同一个圈合并，
 - 2.圈内的1的个数必须是2的幂，
 - 3.画的圈越大越好。
 - 4.边缘可以卷曲，和对面相邻
 - 5.不会出现的项即无关项（don't care）用x表示，可以当1或者0来使用。比如8421BCD码中大于9的那几个项

4.七段数码管（7-segment display）

四·组合逻辑电路

1.加法器adder

1. 半加器：两个输入位相加，得到和& 进位
进位输出 $Cout=AB$ （与门）
和输出 $Sum=A\oplus B$ （异或）
by与门+异或门
2. 全加器：两个半加器基础上增加一个或门/半加器端子上增加输入进位（Cin）
进位输出 $Cout=AB+(A\oplus B)Cin$
和输出 $Sum=(A\oplus B)\oplus Cin$ （异或）
by两个半加器+或门
3. 串行(Serial)加法器：全加器+移位寄存器
4. 并行(Parallel)加法器：全加器+加法器阵列
5. 异步进位加法器/Ripple Carry Adder

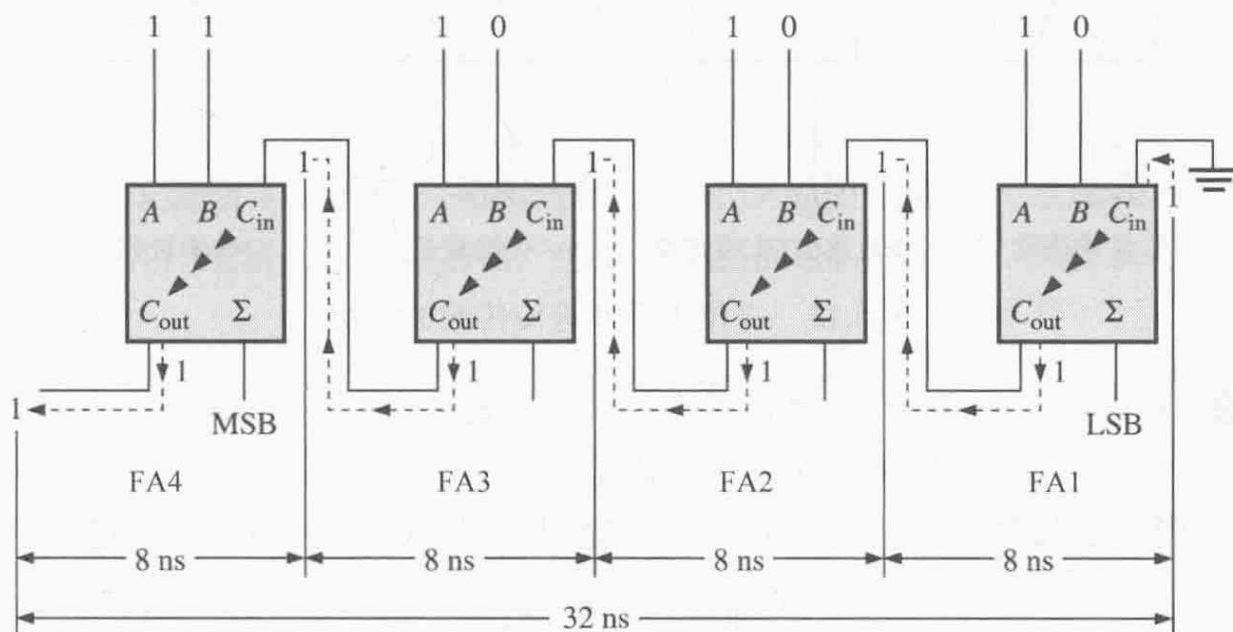


图 6.14 4 位并行异步进位加法器的进位传输延迟的“最坏情况”

6. 超前进位加法器/Look-Ahead Carrier

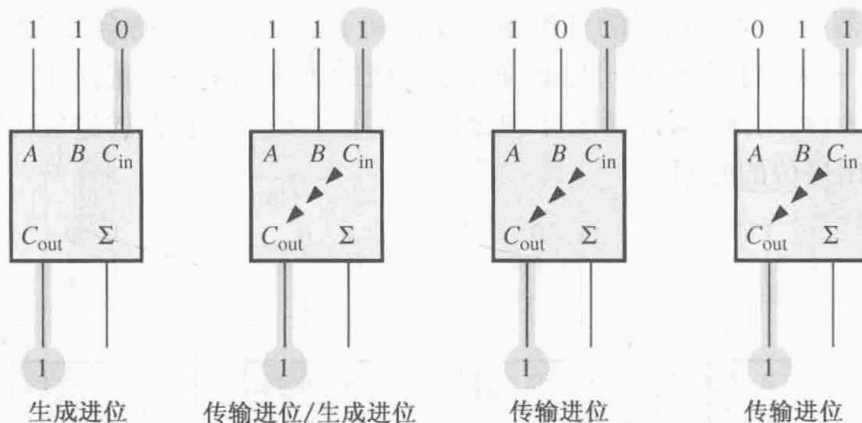


图 6.15 进位生成和进位传输的情况

7. 8421BCD码加法器：两个加法器

2. 比较器comparator

1. 基本比较器 (Only 判断是否相等, 不能比大小): by同或门

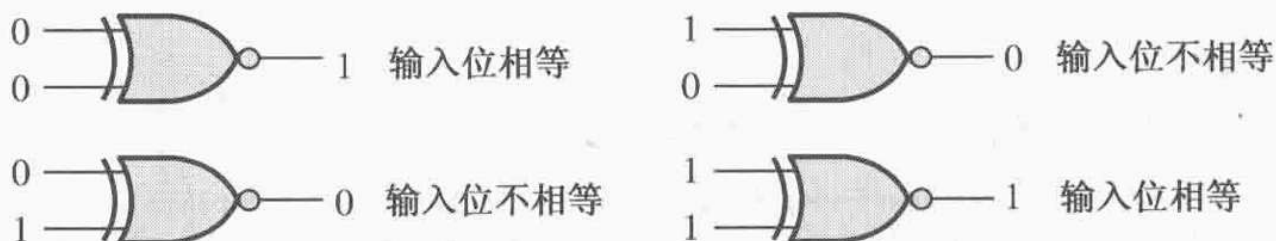
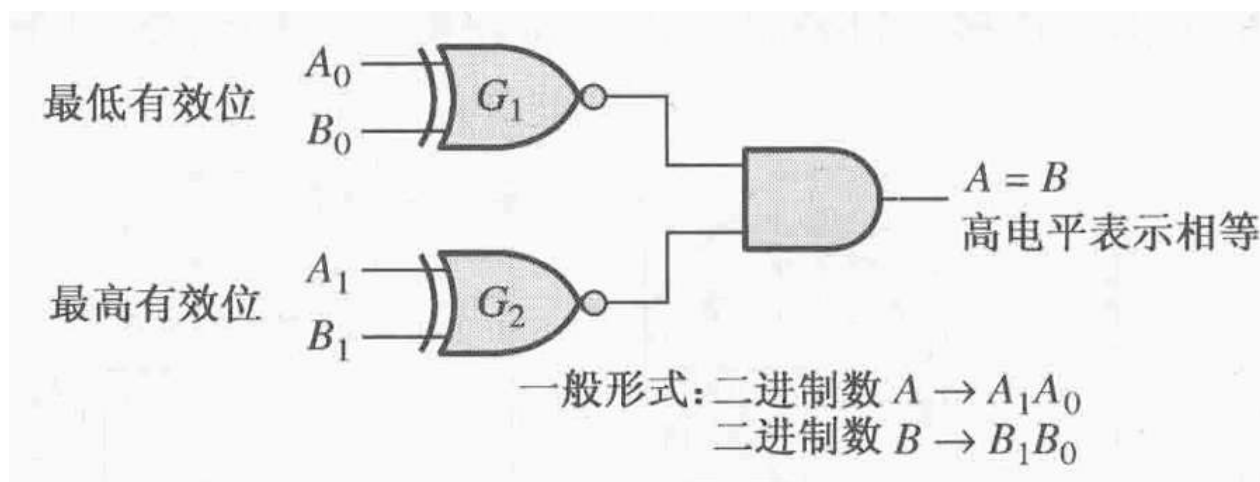


图 6.18 基本比较器的运算



2. 多位比较器：从高位到低位逐位判断，只有高位相等才比较下一位

3. 译码器decoder：检测输入是否为特定组合【重点】

- 分active high/low两类（与非门输出或G'表示active low，与门输出或G表示active high）
- 暴力翻译，通过多组译码器解译所有可能的inputs，给出对应的output。
- 电路图上有一个En的输入线，表示enable（使能信号线），即总开关
- 元件图上if有小三角/圈：active low，反之亦然
- if active high: $y_i = m_i$ ，反之取非
- Warning：引脚不能空置，必须给一个signal，否则会出错
- 画图题中，无译码器限定时，首选3-8译码器
- 译码器的级联（cascading）：
 - 关于所需译码器的数量：3-8 to 4-16：2个；3-8 to 5-32：3个
 - 由于输入不能增多，因此级联后输入端子不够用，就用enable端子来拼

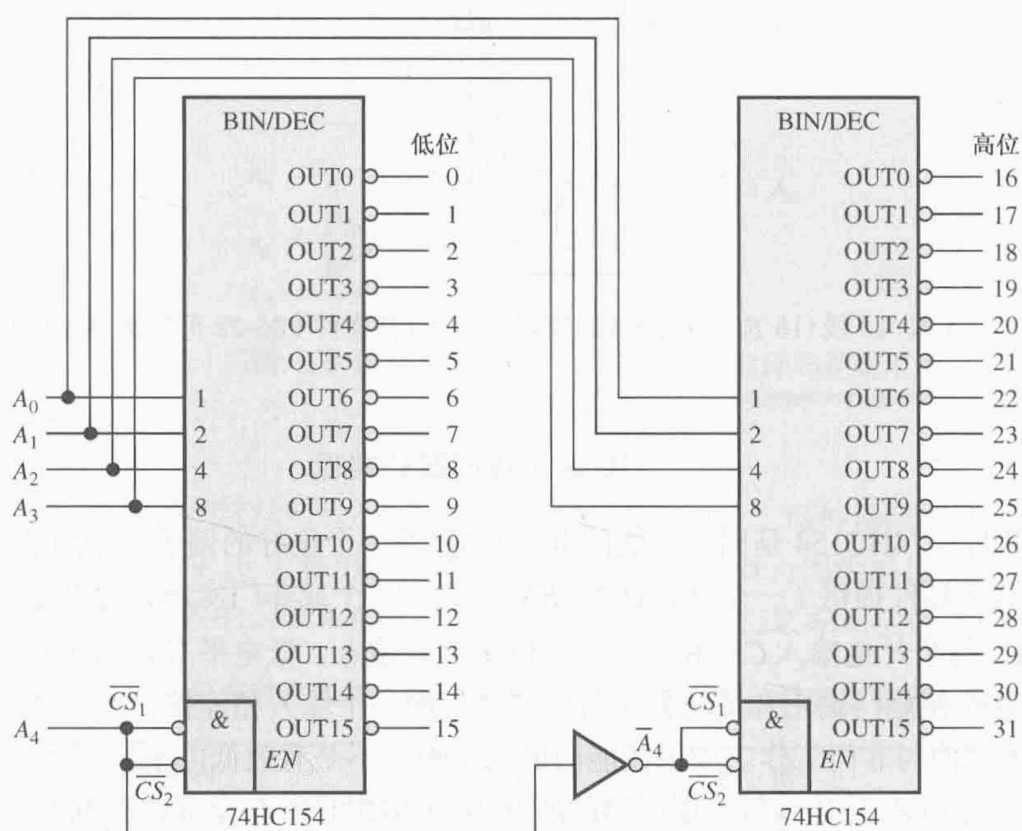


图 6.29 使用 74HC154 的 5 位译码器

- e.g. if $A_4=0, A_3 \sim A_0=1111$, so 15号端子输出0，其余为1(due to active low)
- e.g. if $A_4=1, A_3 \sim A_0=0000$, so 16号端子输出0，其余为1(due to active high)
- A_4 在此处起到片选信号的作用（choose谁工作），驱动两个4-16 single译码器
- 译码器实现布尔函数
 - 布尔函数可表示为最小项之间的或运算，每一根输出对应一个最小项。如果是active-low,在译码器输出后接与非(非或)门; else过或门

- 布尔函数可表示为最大项之间的与运算，再在译码器输出后接与门(非或非)
- 注意顺序：比如 $F(W,X,Y,Z)$ ，W必须连接译码器input端的最高位（一般是最下面的那个D）

4.编码器encoder：把输入编码为特定输出（和译码器相反）

1. 普通编码器：同一时刻只能有一个有效input，无法并行工作
2. 优先编码器/Priority Encoders：同一时刻可以有多个有效input，优先级由输入顺序决定，先编码优先级高的变量
3. 代码转换器：一种编码转换为另一种编码

5.数字多路器/多路复用器multiplexer/MUX：从多个input中选择一个，进行output【重点】

- 2^n 条输入，n条选择信号线，1条输出，1条enable
- 电路图见英文书P201，中文书P197
- 多路器实现布尔函数：
 - 如果选择信号线够用：直接用即可。举个例子，假设 $ABC=010$ 的时候输出应该为0，把对应的第三根输入信号线接地即可。因为此时MUX会链接到这跟信号线上。
 - 如果选择信号线不够用：假设3变量布尔函数，但用的是四选一多路器(四选一只能处理两变量)：两个变量选择信号，剩下的一个变量占用输入,具体参考下图：

IMPLEMENTING BOOLEAN FUNCTIONS WITH MULTIPLEXERS

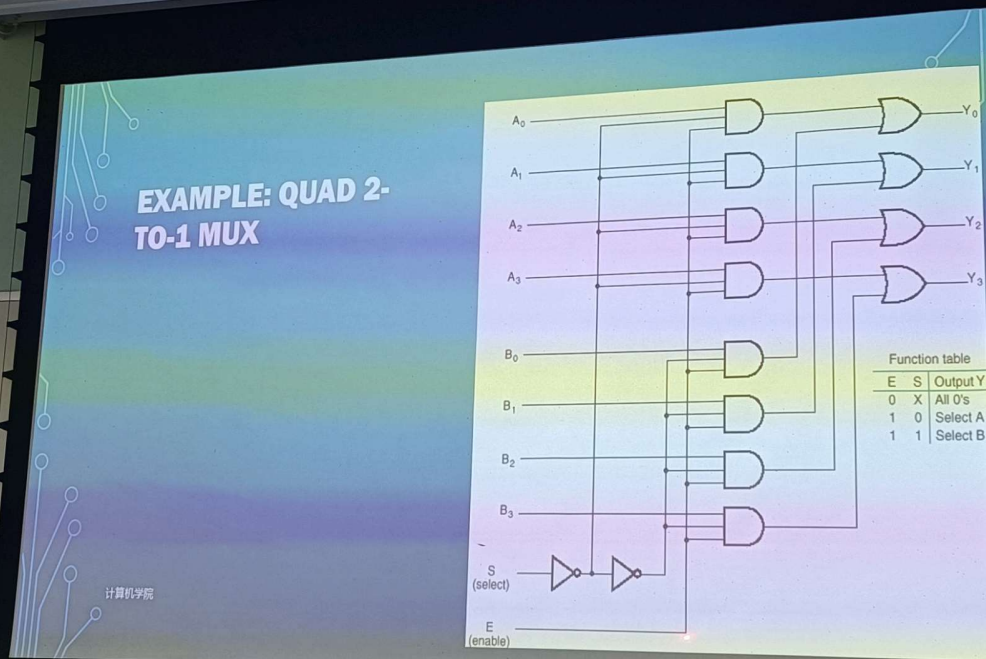
- Using an 4-to-1 multiplexer to realize the Boolean function
 $F=f(x,y,z)=\sum(1,2,4,5,7)$
- $F=f(x,y,z)=\sum(1,2,4,5,7)=x'y'z+x'yz'+xy'z'+xy'z+xyz$
- Let $B=x, A=y$

$D_0=z, D_1=z'$
 $D_2=1, D_3=z$

z \ xy	00	01	11	10
0	0	1	0	1
1	1	0	1	1

$F = x'y'D_0 + x'yD_1 + xy'D_2 + xyD_3$

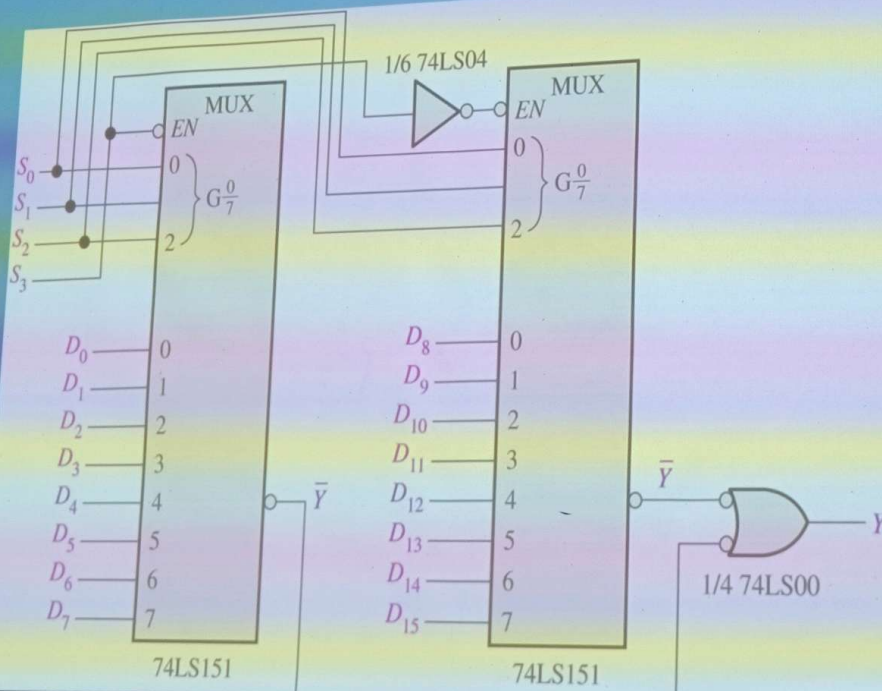
- 选择信号线的顺序是C-B-A（或者 $S_2-S_1-S_0$ ）对应输出端子上的编码。比如在3输入的情况下，110就是CBA'，101就是CB'A。
- 多路器的扩展1：更多输入更多输出，if需要处理多bit信号，采取并行parallel扩展，选择信号和enable共用



4个二选一多路器拼成4位二选一多路器；解释：当S选择信号线为0时，A输入可通过，B输入被与门来自S的0屏蔽，反之亦然。

- **多路器的扩展2：**更多输入，输出数量不变，分高低位芯片（需要多个芯片，注意题目中有无关于芯片数都限制），串行扩展

MULTIPLEXER EXPANSIONS



A 16-1 multiplexer
using two 74LS151

3/31/2025

21

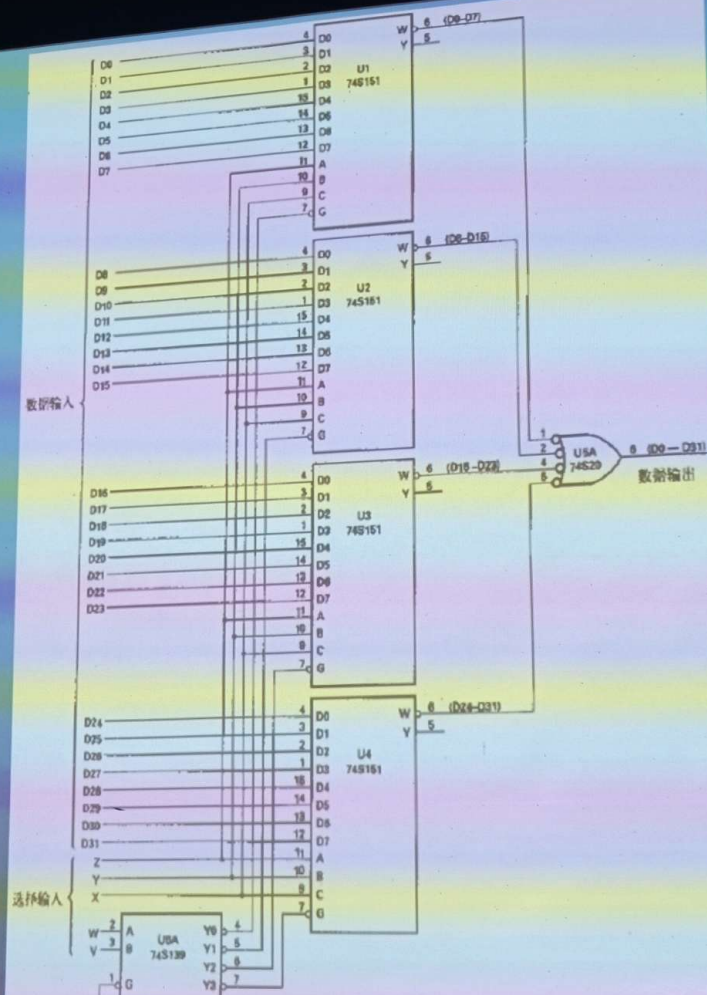
2个八选一拼成一个十六选一，低位选择信号线共享，多的那一根高位选择性号线 S_3 接两边的enable，负责片选信号，保证两个八选一中只有一个会工作。

- 多路器的扩展综合

MULTIPLEXER EXPANSIONS

A 32-to-1 multiplexer using four 8-to-1 multiplexers and a 2-to-4 decoder

计算机学院



22

用4个八选一（并行）和1个2to4译码器（串行）组合成一个三十二选一多路器。

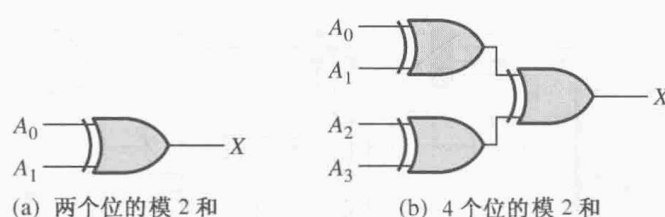
- 多路器用作通用逻辑门(Universal Gate):
 - single与非门(NAND)
 - single或非门(NOR)
 - 与门(AND)+非门(NOT)
 - 或门(OR)+非门(NOT)

6.多路分配器demultiplexer/DEMUX: 从单个input通过特定选择方式到多个output端子

- 奇偶发生器/校验器:

偶数个 1 相加(丢弃进位)的结果始终是 0, 奇数个 1 相加的结果始终是 1。

因此,要确定一个给定的代码具有偶校验还是奇校验,只需把代码中所有的位相加即可。如图 6.53 (a) 所示, 一个异或门可以产生两个位的模 2 和; 如图 6.53 (b) 所示, 3 个异或门相连, 可以产生 4 个位的模 2 和; 以此类推。当输入中 1 的个数为偶数时, 输出 X 为 0(低电平); 当输入中 1 的个数为奇数时, 输出 X 为 1(高电平)。





9 位奇偶发生器/校验器的实现

固定功能的芯片 74HC280 的逻辑符号和函数表如图 6.54 所示。这种特殊的芯片可以用来对一个 9 位代码(8 个数据位和一个校验位)进行奇校验或偶校验,或者还可以为一个 9 位二进制代码产生一个校验位。其中 $A \sim I$ 为输入;当输入端有偶数个 1 时, Σ 偶数输出为高电平, Σ 奇数输出为低电平。

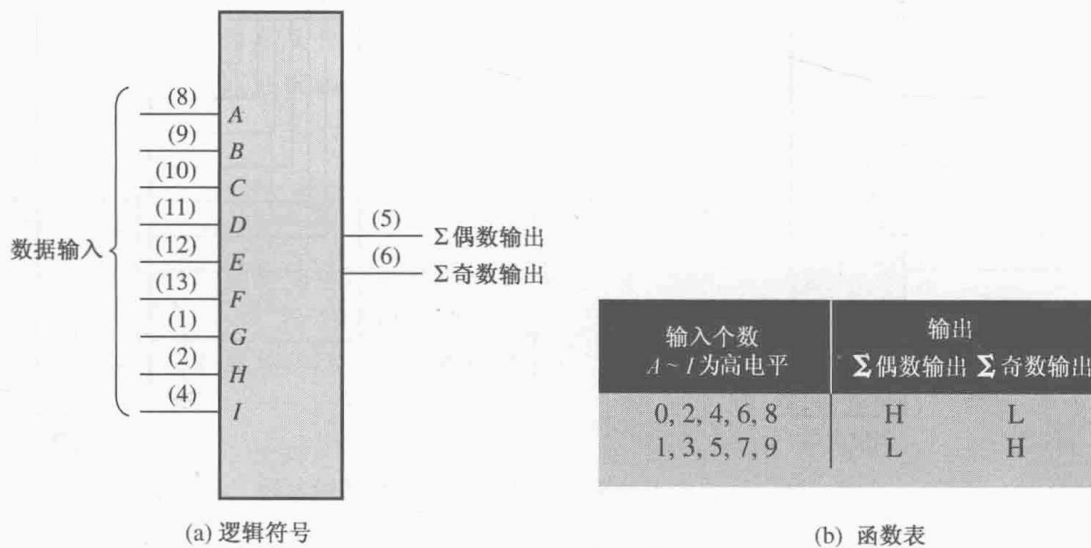


图 6.54 74HC280 9 位奇偶发生器/校验器

奇偶校验器 当把这种芯片用作一个偶校验器时,输入位的个数应该始终为偶数;当出现奇偶错误时, Σ 偶数输出为低电平, Σ 奇数输出为高电平。当把这种芯片用作一个奇校验器时,输入位的个数应该始终为奇数;当出现奇偶错误时, Σ 奇数输出为低电平, Σ 偶数输出为高电平。

五·时序电路（元器件带记忆，存在状态变量state variable）

0.常用表示符/基本概念

- 状态方程: $S_{t+1} = f(E, S_t)$
 - S_t 代表本状态,即现态
 - S_{t+1} 代表下一时刻的状态,即次态
 - 现态和次态以脉冲到达为分界线,和时间变化有关
 - E 指激励,对应会有一个激励方程
- 同步 (Synchronous) 时序电路: 所有变量的变化都跟着时钟的拍子走
- 异步 (Asynchronous) 时序电路: 每个变量不参考同一个时钟信号源,随时可以发生变化 (相比同步更难分析,但效率更高)

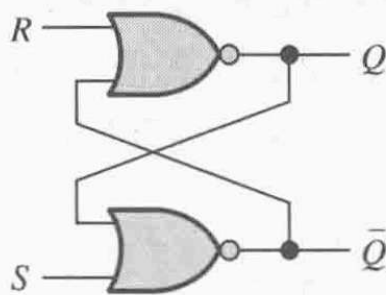
1.锁存器Latch/触发器Filpflop(把他俩当一个东西吧)

- 概念: 临时的存储设备,有两个稳态 (0/1),基本存储元器件,在输入消失时仍然可保持输出。每个门的输出都接到另一个门的输入上。
- 种类: SR, JK, D, T
 - 1. S-R(Set-Reset)Latch:** 用或非门(active-high)/与非门(active-low)构造。特征方程: $Q^{n+1} = S + R'Q^n$, 这也就解释了为什么 active low 要用 S' 和 R' 表示。
 - active low:** 使用或非门,利用 S' 和 R' 表达。
 - Stable Condition: 当两个输入 S' 和 R' 都为高电平 (HIGH) 时, SR 锁存器处于稳定状态 (latched condition)。
 - Set Operation: 要将锁存器置位 (SET), 即输出 $Q=1$, 需要让 $S'=LOW$, 同时保持 $R'=HIGH$ 。
 - Reset Operation: 要将锁存器复位 (RESET), 即输出 $Q=0$, 需要让 $R'=LOW$, 同时保持 $S'=HIGH$ 。
 - active high:** 使用与非门,利用 S 和 R 表达。
 - Stable Condition: 当两个输入 S 和 R 都为低电平 (LOW) 时, SR 锁存器处于稳定状态 (latched condition), 维持先前状态不变。

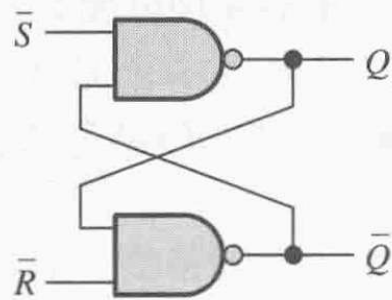
- Set Operation: 要将锁存器置位 (SET)，即输出 $Q=1$ ，需要让 $S=HIGH$ ，同时保持 $R=LOW$ 。
- Reset Operation: 要将锁存器复位 (RESET)，即输出 $Q=0$ ，需要让 $R=HIGH$ ，同时保持 $S=LOW$ 。

SR LATCH

- The active-LOW $S'-R'$ latch is in a stable (latched) condition when both inputs are HIGH.
 - To SET the latch ($Q = 1$), Let $S'=LOW$, $R'=HIGH$
 - To RESET the latch ($Q = 0$), Let $S'=HIGH$, $R'=LOW$
- The active-HIGH S-R latch is in a stable (latched) condition when both inputs are LOW.
 - To SET the latch ($Q = 1$), Let $S=HIGH$, $R=LOW$
 - To RESET the latch ($Q = 0$), Let $S=LOW$, $R=HIGH$



(a) 高电平有效输入S-R锁存器



(b) 低电平有效输入 $\bar{S}$ - $\bar{R}$ 锁存器

图 7.1 S-R 锁存器的两个版本 (打开文件 F07-01 可验证锁存器的操作)

S'-R' LATCH

R'	S'	Q^n	Q^{n+1}	Comments
0	0	0	Invalid	
		1		
0	1	0	0	Reset (置0)
		1		
1	0	0	1	Set (置1)
		1		
1	1	0	0	Remain (保持)
		1		

计算机学院

3/31/2025 13

卡诺图:

	$R'S'$	00	01	11	10
Q^n	0	d	0	0	1
	1	d	0	1	1

约束方程:

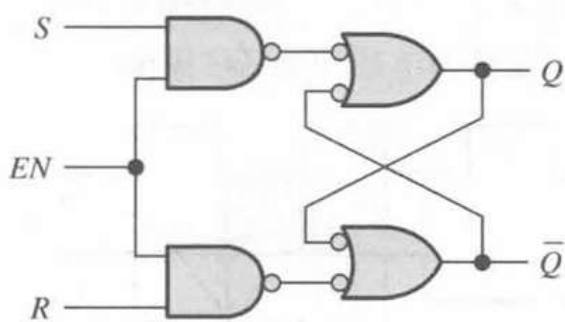
$$\begin{cases} Q^{n+1} = S + R'Q^n \\ S' + R' = 1 \text{ (约束)} \end{cases}$$

SR不能都给有效电平信号,即 $S'+R'=1$, 否则非法

上图中 Q^n 和 Q^{n+1} 分别指现态和次态

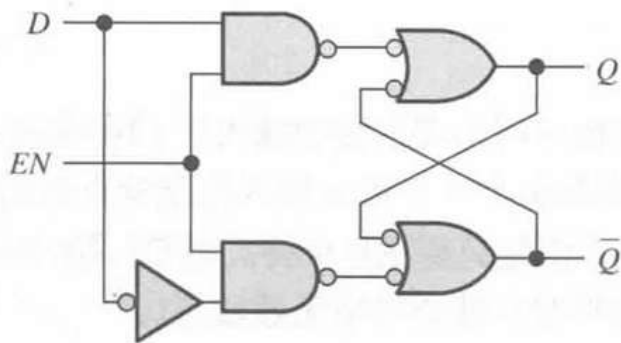
Q' 表示Q的反相, 即 $Q=0$ 时 $Q'=1$, $Q=1$ 时 $Q'=0$, 始终和Q相反, 和 Q^{n+1} 是两个东西。

- 1.5. 门控SR latch: 使用一个使能信号E, 当E=1时, SR latch正常工作, 当E=0时, SR latch不工作, 保持原来的状态。



(a) 逻辑框图

- 2. D Latch: 次态完全由现态决定。D为输入, Q为输出。当使能信号有效时, Q跟随D变化 (完全一样); 当使能信号无效时, Q保持先前状态不变可以确保不会出现SR的非法情况。特征方程: $Q=D$



(a) 逻辑框图

- 3. J-K latch: J为置位信号, K为复位信号。特征方程: $Q^{n+1} = J'Q^n + K'Q^n$ 。

表 7.3 上升沿触发的 J-K 触发器的真值表

输 入			输 出		说 明
J	K	CLK	Q	$\bar{Q}$	
0	0	↑	Q_0	$\bar{Q}_0$	没有变化
0	1	↑	0	1	复位
1	0	↑	1	0	置位
1	1	↑	$\bar{Q}_0$	Q_0	切换

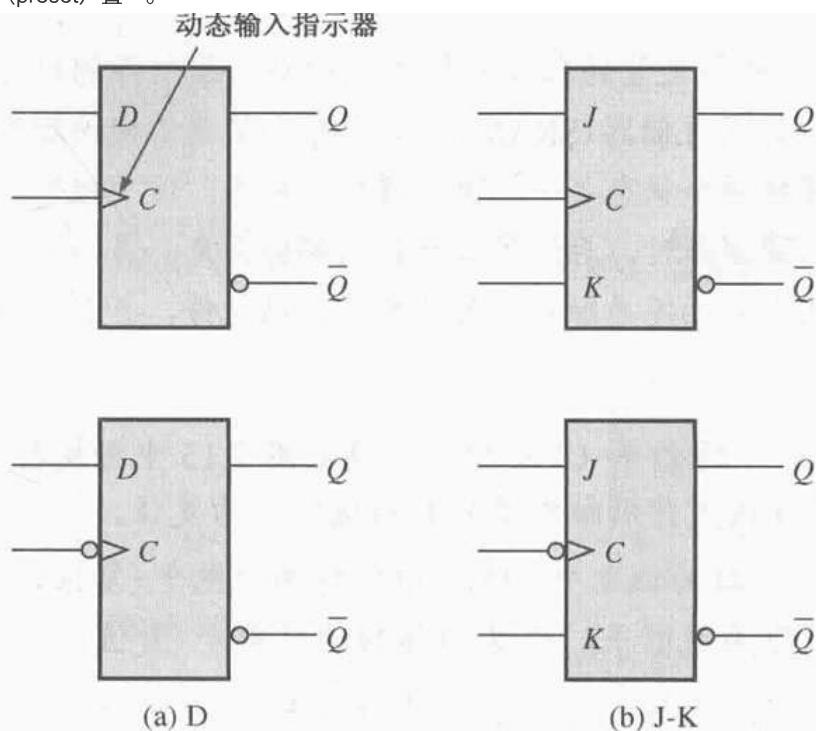
↑ = 低电平到高电平的时钟转换瞬间

Q_0 = 时钟转换瞬间之前的输出电平

(切换/翻转: toggle)

- 边沿触发edge tri: 时钟设备, 在时钟边沿才被触发, 分上ggering (解决不停翻转的问题): 见触发器

- 4. Flip Flop触发器 (基本) / 正边沿(positive)和负/下边沿(negative edge)。后者有小圈。可能会有其他接入端子: CLR (clear) 清零, PRE (preset) 置一。



(a) D

(b) J-K

边沿触发器的逻辑符号(顶部: 上升沿触发的; 底部: 下降沿触发的)

- 5. T flip flop: 可以理解为JK同时接到一个T信号。T为输入, Q为输出。特征方程: $Q^{n+1} = T'Q^n + T'Q^n$ 。T为0保持不变(hold), T为1翻转(toggle)。

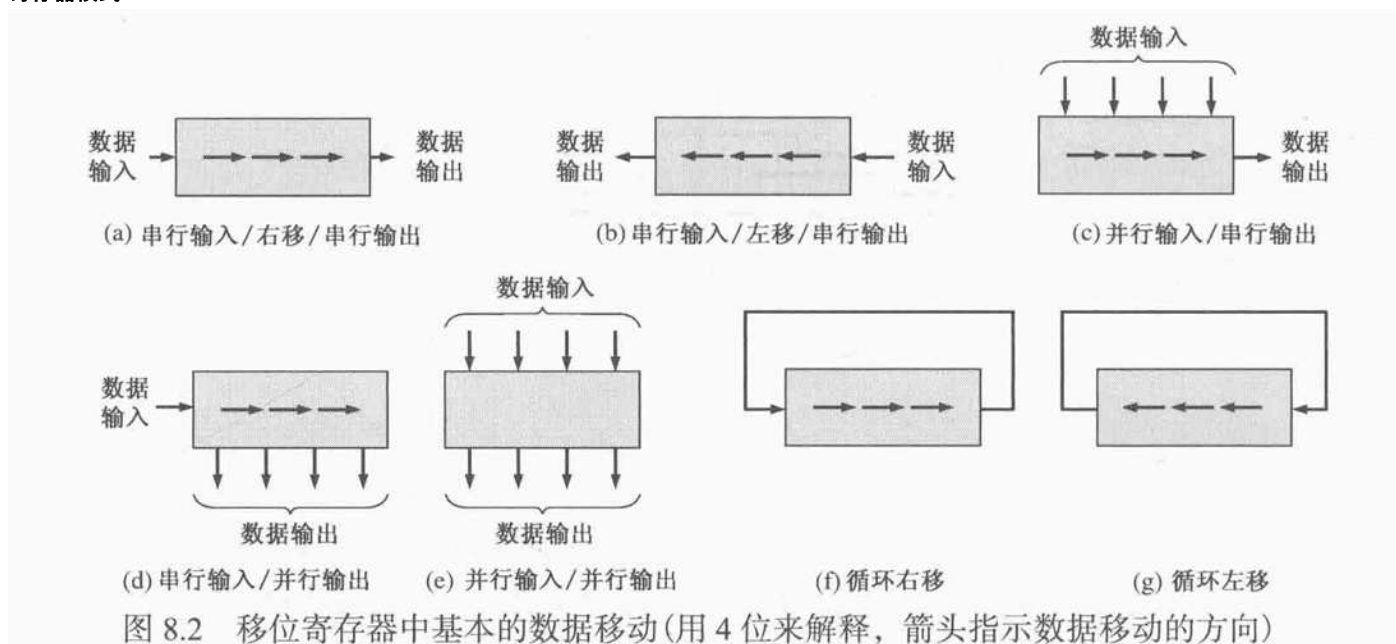
特征方程汇总:

触发器种类	特征方程	特性
SR Latch	$Q^{n+1} = S + R'Q (SR = 0)$	置1/置0/保持/非法
D Latch	$Q^{n+1} = D$	置1/置0
J-K Flip-Flop	$Q^{n+1} = JQ' + K'Q$	置1/置0/保持/翻转
T Flip-Flop	$Q^{n+1} = T'Q + TQ'$	保持/翻转

2.寄存器Registers

1. **寄存器定义**：由多个触发器组成，用于存储二进制数据。每个触发器存储一个bit，寄存器可以存储多个bit的数据。

2. **寄存器模式**：

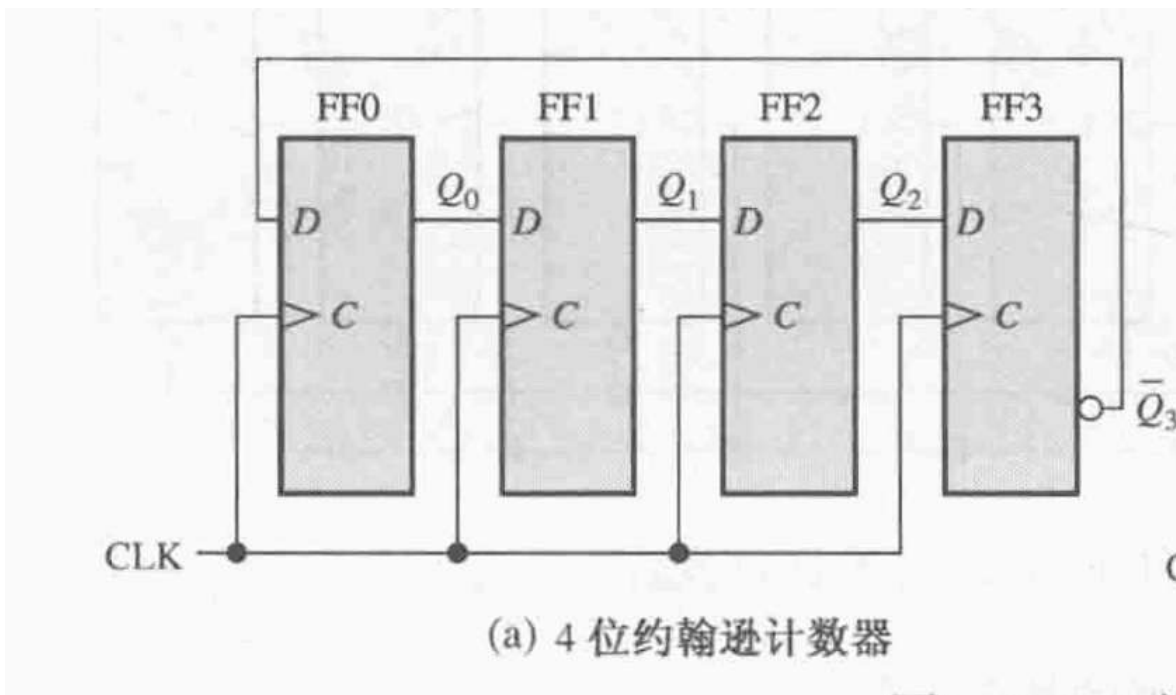


- a. 串入串出右移/SISO/Shift right(LSB->MSB, 不是根据几何方向来判断的)
- b. 串入并出左移/SISO/Shift left(MSB->LSB)
- c. 并入串出/PISO: 中文书P264
- d. 串入并出/SIPO: 中文书P263
- e. 并入并出/PIPO: 中文书P267
- f. 循环寄存器

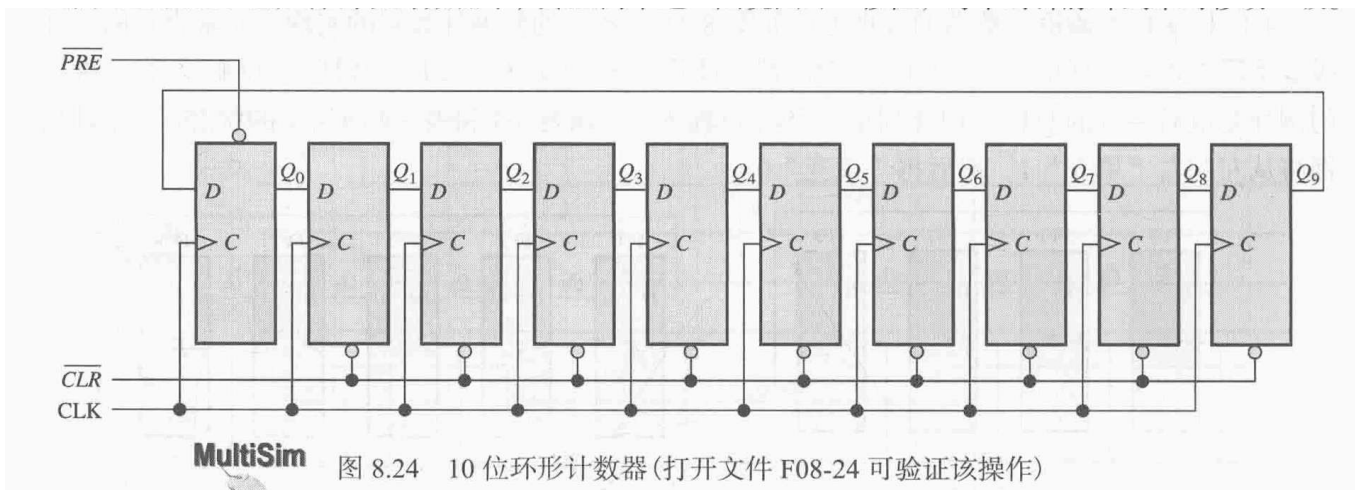
3. **组成**：将flip flops进行cascade，每个flip flop的输出接到下一个flip flop的输入，形成一个链状结构。具体是左移还是右移取决于上下两个flip flops谁是高位，谁是低位。

4. **(环形) 移位寄存器/计数器**：把最后一位的输出接到第一位上，形成一个环。一般由D/JK触发器组成。按这个环扭不扭分两种：

- **Johnson Counter**：扭了（Q3'对应D），最后一位的Q'接到第一位的输入。传了一轮后波形upside down。



- 特性：设寄存器位数为 N ，完整循环的频率为 $\frac{f_{\text{时钟}}}{2N}$ ；状态数为 $2 \times N$
- Ring Counter: 没扭（Q9对应D），和Johnson的区别在最后一位的输出，Johnson有非，Ring没有。所以Ring不能初始置0，否则会恒为0。



- 特性：设寄存器位数为 N ，不存在固定频率；状态数为 N
- 可实现self-decoding(即根据当前状态推导当前时钟信号)：
- P.S.如果看不懂，去书上找相关的时序波形图来分析，英文书P273-275
- Sum: Ring: N 个FF- N 个状态；Johnson: N 个FF- $2N$ 个状态

3. 计数器Counters

0. 基本概念

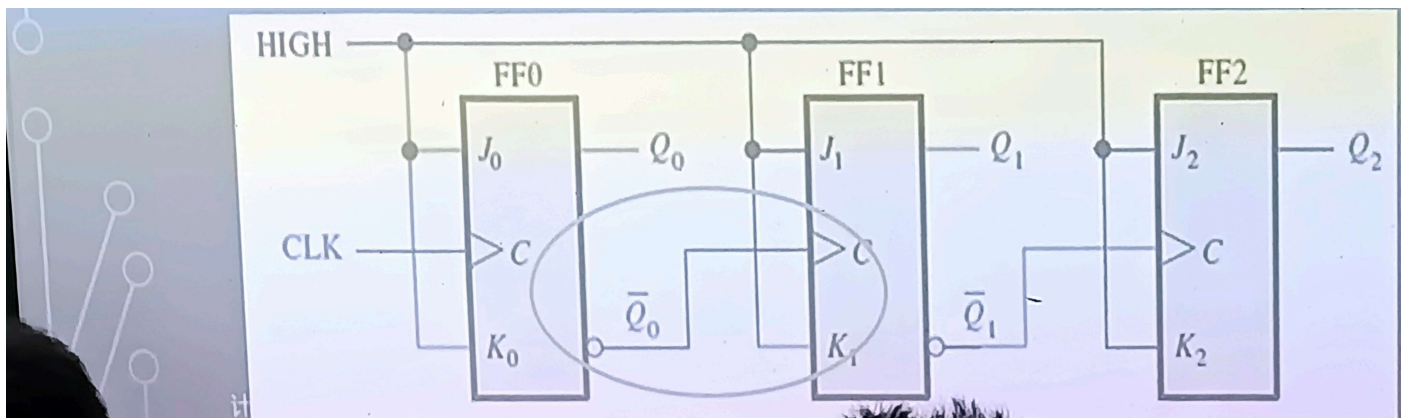
- 分类：
 - Moore型：输出只由现态决定
 - Mealy型：输出由现态和输入决定
- 变量：输入/输出/状态/激励

1. 计数器定义：在时钟脉冲的作用下根据预制输入计数

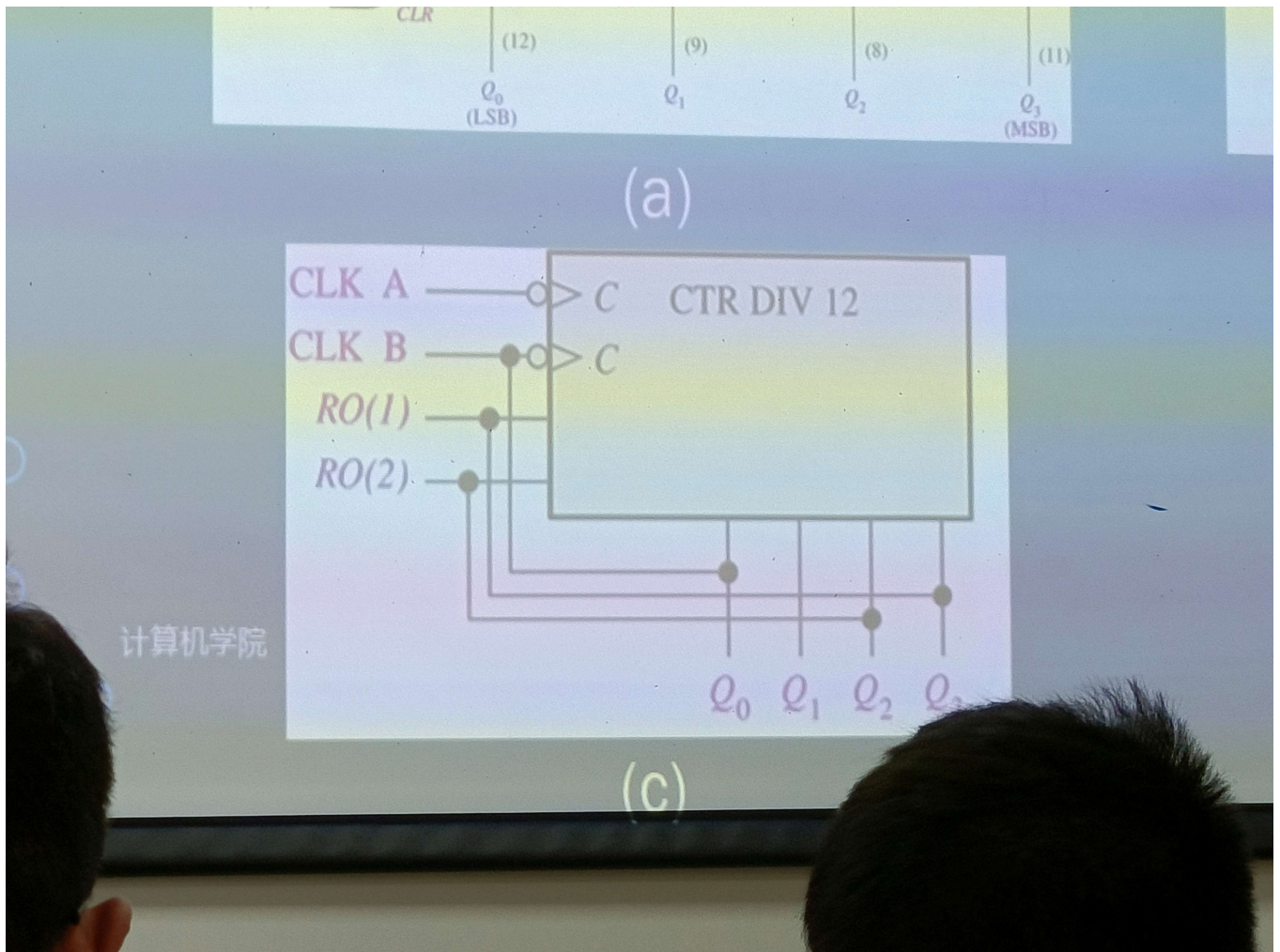
2. 分类：

- 异步计数器Asynchronous Counter：不需要共同时钟，可以把输出用作后续触发器的时钟信号
- 同步计数器Synchronous Counter：需要共同时钟

3. 异步计数器设计



- P.S.J-K全接高电平，使用其toggle翻转功能。上升沿(0→1)时钟接 Q' ，下降沿(1→0)接 Q
- Warning:计数器不能无限cascade flip-flops，因为存在传输延迟
- 计数器的模Modulus：计数器按顺序经过的独特状态数目，也就是计数器完成一次完整计数周期所需的状态数。比如模4计数器：能循环计数0、1、2、3，共4个状态，数到3后回到0。(易错：模不是计数器能呈现的最大数)
- 对模不等于 2^n 的计数器：by截断序列/局部译码：比如10位计数器：全局译码表示每一个flip-flop输出译码为1010时截断。而部分译码表示只care Q_1 和 Q_3 ，当且仅当二者都为1时截断（其他项对应组合中的任意项），通过clear端子操作截断。

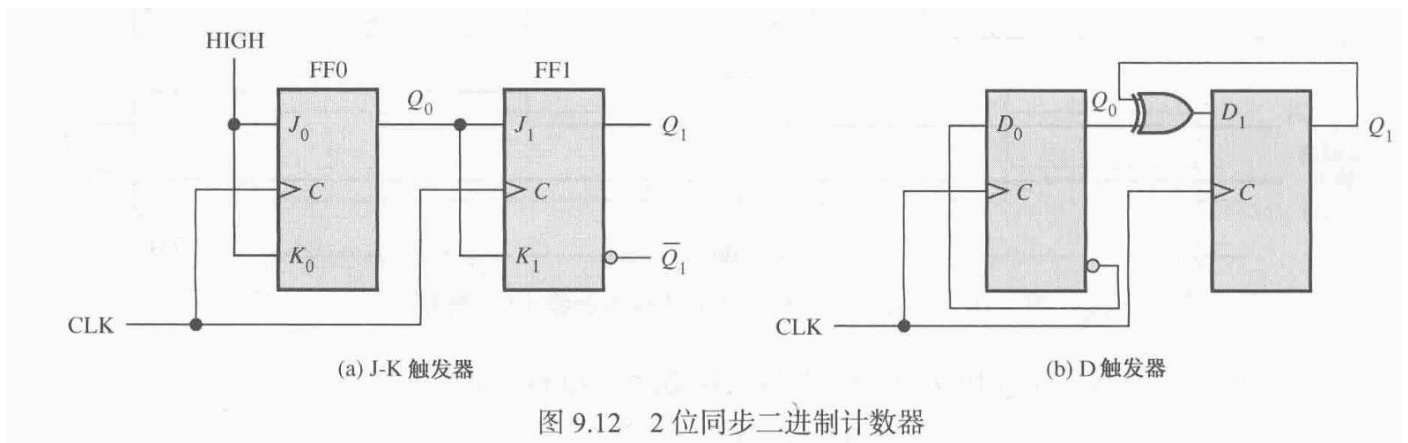


Clock B接 Q_0 ，实现异步计数器；

两根RO线接 Q_2, Q_3 ，表示当且仅当 $Q_2=Q_3=1$ ，即 $output \geq 1100_2 = 12_{10}$ 时截断，即图上是一个模12计数器在截断点会出现一个Glitch（毛刺），不记入模。

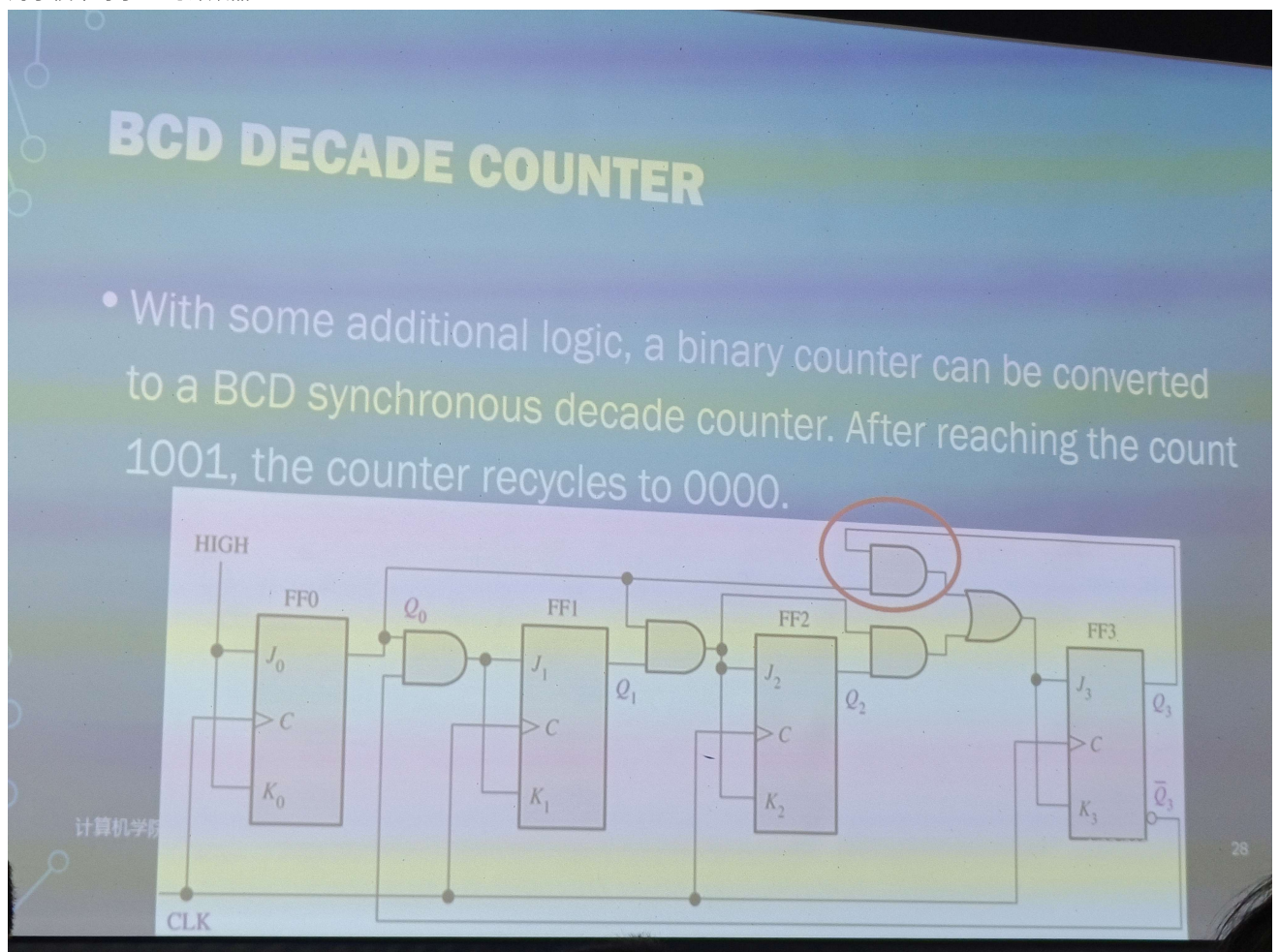
- 通用设计方法：
 - 选择flip-flop的类型，是JK还是D，他们有自己的接线方式。
 - 确定flip-flop的数目： $2^{n-1} < M < 2^n$
 - 让所有触发器在M-1状态能快速归0
 - if $K=2^n$ ：不需要做额外动作
 - if $K < 2^n$ ：阶段序列&局部译码，配合逻辑门实现。

4. 同步计数器设计：没有传输延迟



• 通用设计方法：

- 对于模 2^n 的计数器：
 - 2位计数器：上一片flip-flop输出为1，下一片在下一个时钟脉冲翻转
 - 3位计数器：前两片输出为11，下一片在下一个时钟脉冲翻转
 - 以此类推.....
- 对于模不等于 2^n 的计数器：



- 圈起来的那个与门译码 Q_0 和 Q_3 ,检测 $1001_2=9_{10}$
- 当Q达到1001时，激活圈起来的与门，后面那个或门恒为1，使最后一位JK翻转，在下一个时钟脉冲中 Q_3 翻转为0；下一个时钟脉冲当中，FF1翻转， Q_1 翻转为0，进而带动FF2和FF3变化，全部归0。

5. 计数器级联Cascade

典例：中文书P314上方

例 9.7 使用两个 74HC190 加/减十进制计数器, 连接为加模式, 从 1 MHz 时钟获得一个 10 kHz 的波形, 给出逻辑框图。

解: 为了从 1 MHz 时钟中得到 10 kHz, 需要除以因数 100。两个 74HC190 计数器必须依照图 9.40 所示的那样进行级联。左边的计数器每 10 个时钟脉冲产生一个终端计数输出 (MAX/MIN)。右边的计数器每 100 个时钟脉冲产生一个终端计数输出 (MAX/MIN)。

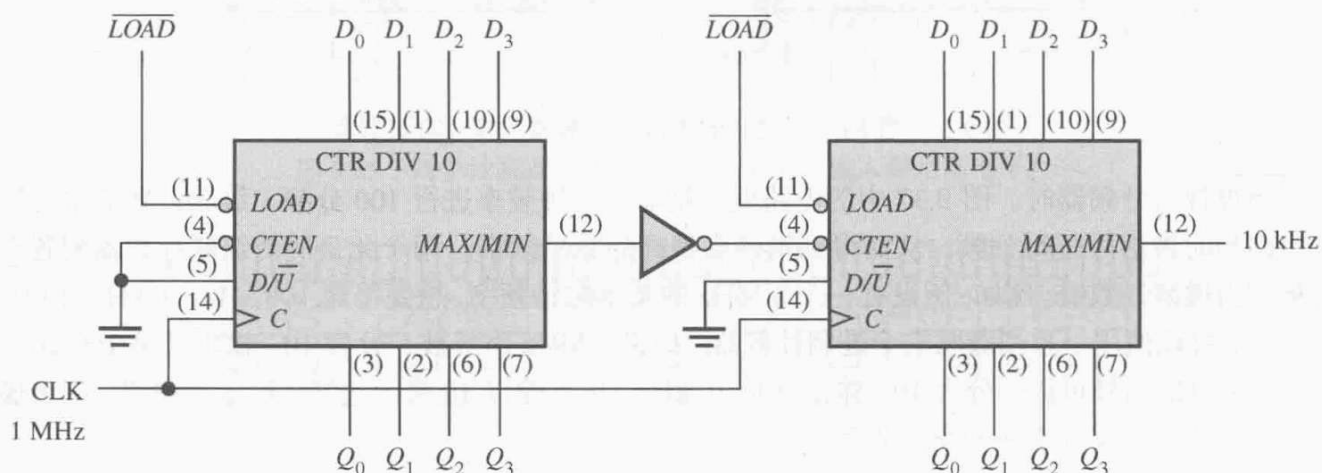


图 9.40 使用两个 74HC190 加/减十进制计数器的 100 分频计数器

图中的两个计数器为模 10 计数器, 级联后为模 100 计数器。

简化后如下图所示:

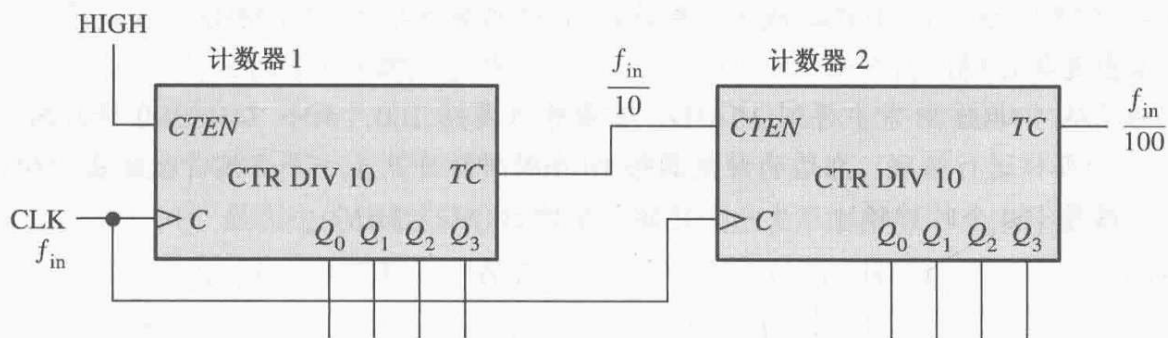
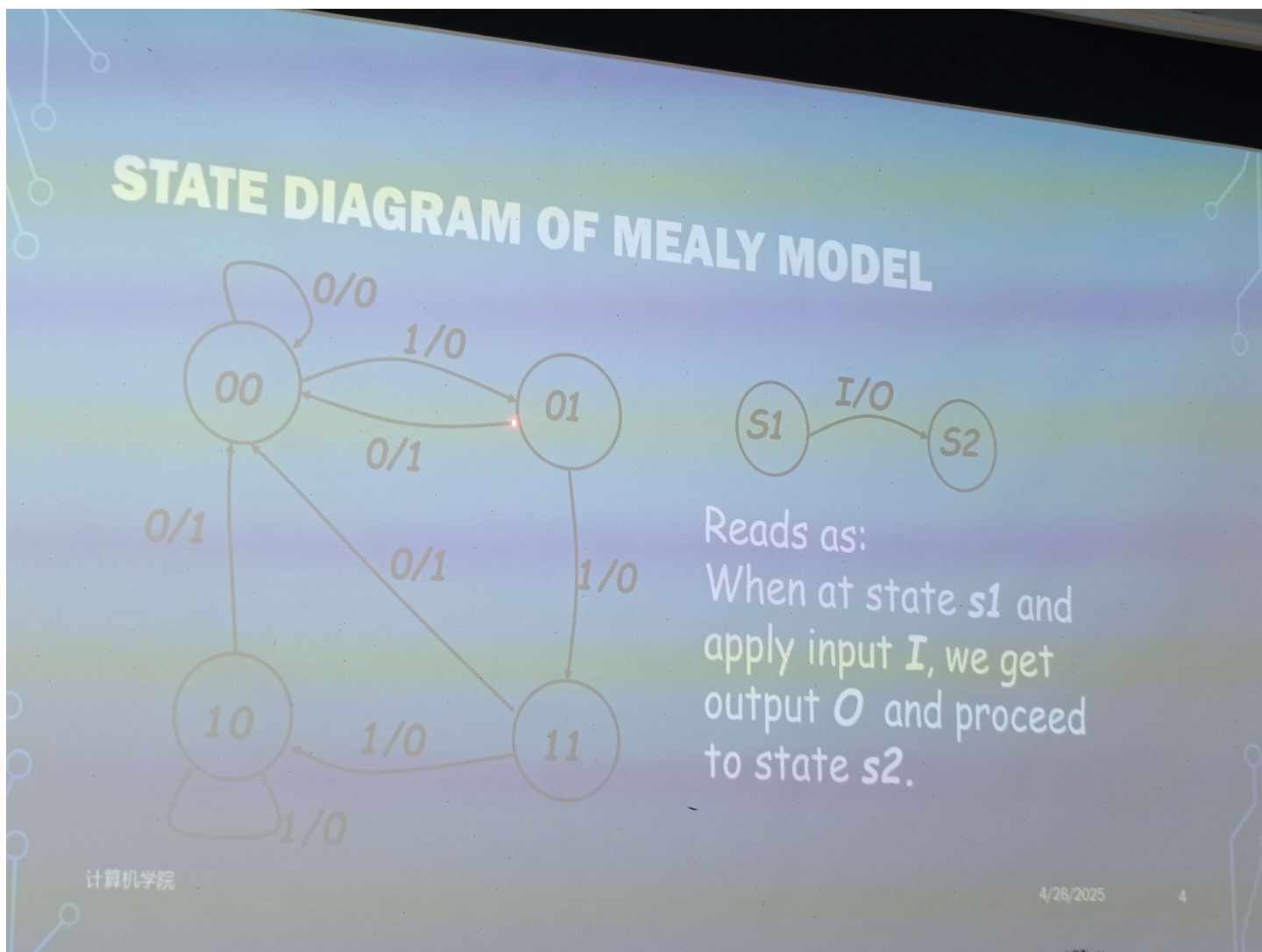


图 9.37 使用两个级联的十进制计数器的模 100 计数器

六.时序电路设计

问题->状态图->状态表->确定FF数量后状态转换表->激励表->卡诺图->激励方程->电路设计, 例题见Chapter 9_02.pptx

1.状态图 State Diagram

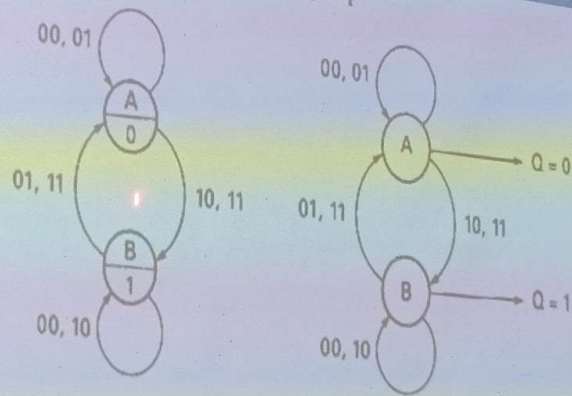


通过这张图，可得：1个输入变量，1个输出变量，2个触发器（对应 $2^2=4$ 个状态）

STATE DIAGRAM

J	K	Next State of Q
0	0	Q
0	1	0
1	0	1
1	1	$\bar{Q}$

Moore circuit notation of a J-K flip-flop



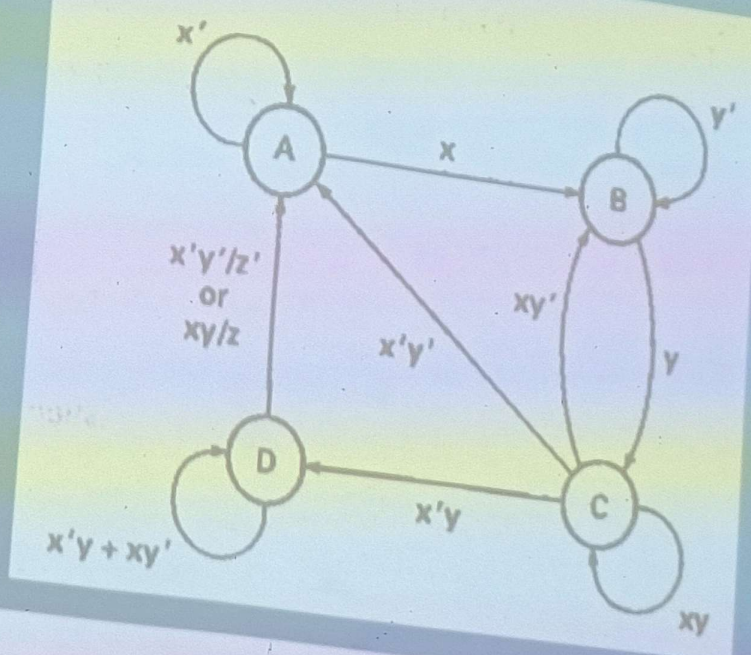
(a) Output Variables
Written under State
Variable Names

(b) Output Variables
Indicated by Arcs

2.状态表 State Table

STATE TABLE

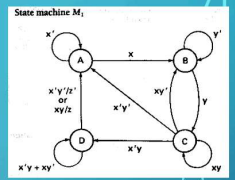
State machine M_1



Present state	Next state xy/z							
	00		01		11		10	
	z		z		z		z	
A	A	0	A	0	B	0	B	0
B	B	0	C	0	C	0	B	0
C	A	0	D	0	C	0	B	0
D	A	0	D	0	A	1	D	0

- 注：只有状态图上有z的时候才会输出1，反之都是0（输出可以用这种形式写在状态图上）
- 再通过**状态转换表Transition Table**，给ABCD状态赋一个编码，对应触发器输出，比如A=00,B=01...。在下表中，z是输出，D是次态

EXCITATION TABLE AND EQUATIONS



Transition Table for state machine M1

Present state $F_A F_B$		Next state xy/z			
		00 z	01 z	11 z	10 z
0 0	0 0	0 0 0	0 0 0	0 1 0	0 1 0
0 1	0 1	0 1 0	1 1 0	1 1 0	0 1 0
1 1	1 1	0 0 0	1 0 0	1 1 0	0 1 0
1 0	1 0	0 0 0	1 0 0	0 0 1	1 0 0

- D flip-flop used to realize circuit (two D flip-flops)

D flip-flop characteristic table and equation

D	Present, Q_t	Next, Q_{t+1}
0	0	0
1	0	1
0	1	0
1	1	1

$Q_{t+1} = D$

D flip-flop excitation table

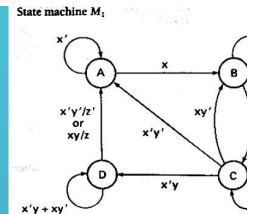
Q_t	Q_{t+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

state machine M1 excitation table using D flip-flops

Present state $F_A F_B$		Next state xy/z			
		00 z	01 z	11 z	10 z
0 0	0 0	0 0 0	0 0 0	0 1 0	0 1 0
0 1	0 1	0 1 0	1 1 0	1 1 0	0 1 0
1 1	1 1	0 0 0	1 0 0	1 1 0	0 1 0
1 0	1 0	0 0 0	1 0 0	0 0 1	1 0 0

- 再选择相应的触发器把上述表对应电路设计出来（参考触发器的激励表Excitation Table），再通过卡诺图化简。

EXCITATION TABLE AND EQUATIONS



D_A K-Map

$F_A F_B$	00	01	11	10
xy	0	4	12	8
00				
01		1	1	1
11		1	1	
10				1

D_A K-Map

D_B K-Map

$F_A F_B$	00	01	11	10
xy	0	4	12	8
00		1		
01		1		
11	1	1	1	
10	1		1	

D_B K-Map

Z K-Map

$F_A F_B$	00	01	11	10
xy	0	4	12	8
00				
01				
11			1	
10				

Z K-Map

$$D_A = f(F_A, F_B, x, y) = \Sigma(5, 7, 10, 13, 15) = F_A F_B' x y' + F_A x' y + F_B y$$

$$D_B = f(F_A, F_B, x, y) = \Sigma(2, 3, 4, 5, 6, 7, 14, 15) = F_A' F_B + F_A' x + F_B x$$

$$Z = f(F_A, F_B, x, y) = \Sigma(11) = F_A F_B' x y$$

六·2·时序电路分析

(例题：Chapter10 PPT example2)

1. 确定输入/输出/状态变量
2. 确定触发器类型，写出触发器的状态方程
3. 读图，写出触发器的激励方程，代入特征方程，写出次态方程
4. 写出输出方程
5. 构造状态转换表（通过上面的表达式写出每个情况下次态和输出的值）
6. 分配助记符（比如六-2中提到的A,B,.....）
7. 画出状态表(相比转换表有助记符)->状态图

8. (题目层面)按要求画出时序图/波形图【注意上边沿/下边沿触发】，还可判断时序电路用途